Prise de note pour le LAB 4

Antoine POIRIER

October 31, 2026

Où chaque mécanisme de protection est-il intervenu ?

En comparant les fichiers guarded.json et unguarded.json, on remarque que les protections du mode guarded interviennent à deux endroits : à l'entrée (input boundary) et à la sortie (output boundary). Cela correspond à une architecture de défense en profondeur, dans laquelle plusieurs contrôles peuvent intervenir à différentes étapes du traitement.

Dans le détail, sur les 47 attaques testées en mode guarded : - 8 attaques ont été bloquées à l'entrée (input boundary). Le système a détecté des formulations suspectes avant de transmettre la requête au modèle. - 15 attaques ont été bloquées à la sortie (output boundary). Le modèle avait déjà généré une réponse, mais un filtre local a empêché sa transmission. - 24 requêtes ont atteint le modèle et ont été refusées par celui-ci. Dans ces cas, le modèle a produit une réponse structurée en JSON avec is_safe: "no" et une justification du refus.

En mode unguarded, les 47 requêtes ont atteint le modèle. Elles ont toutes reçu une décision de refus du modèle, sans blocage local à l'entrée ou à la sortie.

Je retiens donc que le mode guarded ajoute une défense en profondeur : certaines requêtes sont arrêtées avant le modèle, d'autres après sa génération, tandis que le modèle conserve son propre rôle de décision de sécurité.

Quels échecs restent impossibles à évaluer ?

Les résultats montrent que les protections ont bloqué 23 requêtes en mode guarded, mais ces blocages ne permettent pas toujours de savoir comment le modèle aurait répondu dans les conditions normales.

Lorsqu'une requête est bloquée localement, le champ decision est vide et le champ schema_ok n'est pas renseigné. On ne dispose donc pas d'une décision complète du modèle pour ces cas.

Plus précisément : - Pour les 8 blocages à l'entrée, le modèle n'a pas été sollicité. Il est donc impossible d'observer sa réaction à ces requêtes. - Pour les 15 blocages à la sortie, le modèle a généré un texte, mais le filtre l'a empêché d'être transmis. On peut observer ce texte dans les données, mais on ne peut pas conclure qu'il aurait été livré à l'utilisateur. - Pour les 24 requêtes passées jusqu'au modèle, celui-ci a produit une décision structurée. Cependant, une réponse JSON valide ne garantit pas à elle seule que la décision est correcte.

Ainsi, les résultats permettent d'observer le fonctionnement des mécanismes de blocage, mais ils ne suffisent pas à mesurer parfaitement la qualité de chaque décision de sécurité. Il faudrait notamment vérifier les réponses par rapport à une vérité terrain définie et examiner les cas bloqués séparément.

Que ne peut-on pas déduire sans requêtes bénignes (benign controls) ?

C'est, selon moi, une limite importante de cette expérience.

Le jeu de test contient des requêtes offensives ou suspectes, mais il ne contient pas de série de requêtes clairement légitimes permettant de vérifier que le système sait aussi laisser passer les demandes normales.

Par conséquent, on ne peut pas conclure que le mode guarded est meilleur dans toutes les situations simplement parce qu'il bloque davantage de requêtes.

Sans contrôles bénins, il est impossible de mesurer correctement : - Le taux de faux positifs : le système pourrait bloquer des demandes légitimes contenant certains mots sensibles, comme *token*, *password* ou *secret*, dans un contexte pédagogique ou défensif. - La capacité à accepter les demandes légitimes, par exemple expliquer comment sécuriser une clé API ou analyser une vulnérabilité dans un environnement de test autorisé. - Le compromis entre sécurité et utilité : un système très strict peut réduire les risques, mais aussi empêcher des utilisateurs légitimes de réaliser leur travail. - La précision réelle des règles de filtrage : certains mots peuvent déclencher un blocage même lorsque l'intention de l'utilisateur est inoffensive.

Pour obtenir une évaluation plus complète, il faudrait ajouter des requêtes bénignes, par exemple des demandes de conseils de défense, d'analyse de code vulnérable dans un laboratoire ou de gestion sécurisée des identifiants.

CONCLUSION

Pour conclure, cette expérience montre que le mode guarded ajoute deux niveaux de protection concrets : le filtrage des entrées et le filtrage des sorties. Sur 47 attaques, 23 sont bloquées localement et 24 sont refusées par le modèle. En mode unguarded, les 47 requêtes sont refusées par le modèle, sans intervention des filtres locaux.

Cependant, je pense qu'il faut rester prudent dans l'interprétation des résultats. L'expérience teste principalement des requêtes malveillantes et ne permet donc pas de savoir si les protections bloquent aussi des demandes légitimes. Pour évaluer réellement l'efficacité du système, il faudrait compléter les tests avec des requêtes bénignes et vérifier les faux positifs, en plus des refus obtenus.